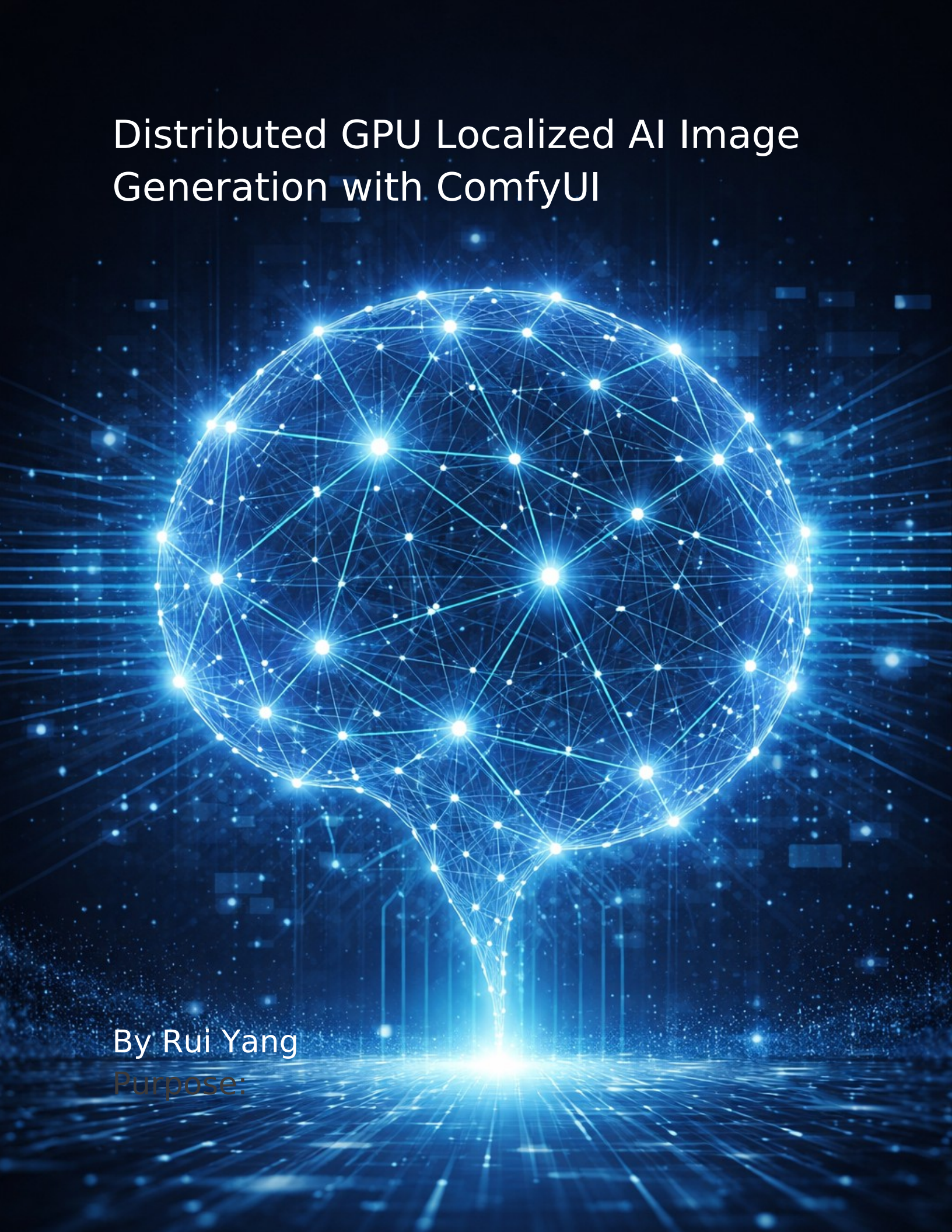


Distributed GPU Localized AI Image Generation with ComfyUI

By Rui Yang
Purpose:



The purpose of this lab is to configure an image generation pipeline using ComfyUI while utilizing multiple GPUs and to access through a local web-based graphical interface. The setup should allow multiple GPUs to work in parallel on generation tasks, increasing the overall productivity.

Background Information:

Artificial Intelligence has recently expanded beyond just simple text-based applications, with image generation emerging as one of the more computationally demanding and rapidly evolving areas of AI development. Modern AI image generation relies on diffusion models, which are sophisticated algorithms that learn to reverse the process of noise addition applied to training images. These models iteratively refine random noise into coherent images by understanding the statistical relationships between image data and textual descriptions. During training, the model is exposed to gigabytes to even terabytes of datasets of images and their associated text descriptions, in which the model learns the statistical relationship between language and visual content. At inference time, the model begins with random noise and iteratively refines it into a coherent image while being guided by a prompt. This denoising process is computationally intensive and is handled primarily by a component called the U-Net, which predicts and removes noise at each step. Supporting components such as the text encoder, which interprets the prompt, and the Variational Autoencoder (VAE), which compresses and reconstructs image data, work alongside the U-Net to produce the final output.

Because each step of the diffusion process involves large matrix operations across high-dimensional data, image generation is extremely well suited to GPUs. Unlike CPUs, which are optimized for performing tasks linearly and is often predictable and reliable, GPUs contain thousands of smaller cores and are designed to handle many parallel operations simultaneously. This allows GPUs to perform the same computations much faster than a CPU alone. The dedicated memory on a GPU, known as VRAM, is extremely crucial, as the model weights, intermediate activations, and image tensors should all reside in VRAM during generation. Larger or higher-resolution models demand more VRAM, and when a single GPU's capacity is insufficient, the only two options are to either offload to system ram or distributing the workload across multiple GPUs. However, offloading VRAM usage to system ram drastically decreases the rate and depending on how much physical RAM is being used, processing speed could go to a fraction of the speed, up to 20-50 times

slower than if the processing was being done fully by VRAM. Because of this, distributing workload across multiple GPUs has become a practical and necessary solution.

Distributed GPU computing in the context of image generation with AI allows multiple graphics cards to each independently process separate generation jobs simultaneously, rather than splitting a single job across cards. This approach, sometimes called data parallelism, drastically increases the speed of tasks that require batch generation where many images need to be produced in a short period of time. Each GPU operates as its own inference node, receiving a unique seed value and running the full generation pipeline independently, with results collected and aggregated at the end. Managing this kind of multi-GPU workflow manually would require a more complex setup, but tools like the ComfyUI-Distributed plugin abstract much of this complexity behind a node-based visual interface, allowing users to configure multi-GPU pipelines without having to write custom code.

ComfyUI itself is an open-source, node-based graphical interface for constructing and executing AI image generation workflows. Rather than exposing a simple prompt-and-generate interface, ComfyUI allows users to build pipelines by having distinct steps of AI computing into blocks and allows the user to connect these individual processing nodes such as model loaders, samplers, encoders, and output formatters. This gives the user fine-grained control over every stage of the generation process. This flexibility makes it well suited for advanced configurations like distributed generation, where additional nodes can be inserted into an existing workflow to route tasks across multiple compute resources with minimal changes to the pre-existing pipeline.

Lab Summary:

In this lab, we configured a multi-GPU distributed image generation environment using ComfyUI and the ComfyUI-Distributed plugin. We installed the plugin into ComfyUI's custom nodes directory, created a YAML configuration file to provide the CLI runtime with access to the application's model library, and authored batch scripts to launch a master and worker instance of ComfyUI on separate ports with each bound to a distinct GPU via CUDA device assignment. Using the distributed nodes provided by the plugin, we modified an existing image generation workflow to route parallel generation tasks across both GPUs simultaneously, with a collector node

aggregating the outputs into a unified result. The completed setup enables accelerated batch image generation entirely on local hardware, with no internet dependency, and is accessible through ComfyUI's web-based interface on the local network.

Steps and Configuration:

Note: Before anything else, locate your ComfyUI installation folder. It should be somewhere along the lines of C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\.

You'll need to find this folder because it's where you'll be installing the plugin. You'll also need Git installed on your computer. If you don't have it yet, just search "Git for Windows" and follow the instruction provided by the installer.

To start the installation

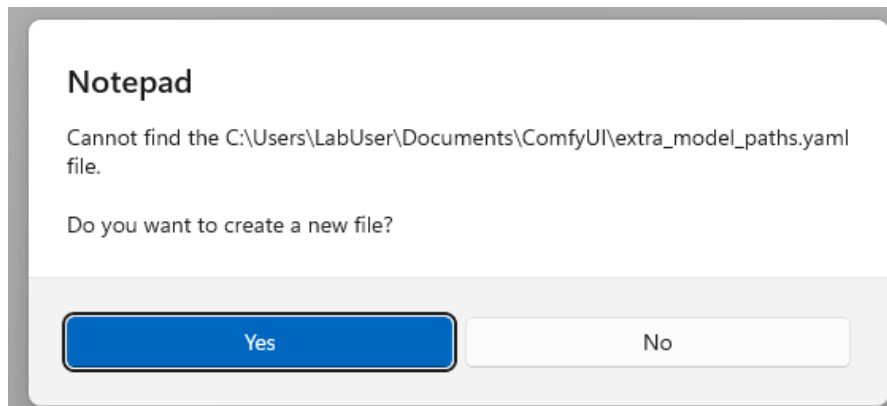
Navigate to the custom_nodes folder inside your ComfyUI installation directory. Once you're there, open a command prompt in that folder and run a "git clone" of the ComfyUI-Distributed repository from GitHub. This will automatically download and install the plugin. More information can be found in https://docs.comfy.org/installation/manual_install#portable%2Fmanual-install

or ComfyUI's GitHub at <https://github.com/Comfy-Org/ComfyUI>

```
PS C:\Users\LabUser> cd C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\custom_nodes
PS C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\custom_nodes> git clone https://github.com/robertvoy/ComfyUI-Distributed.git
Cloning into 'ComfyUI-Distributed'...
remote: Enumerating objects: 1787, done.
remote: Counting objects: 100% (1124/1124), done.
remote: Compressing objects: 100% (446/446), done.
remote: Total 1787 (delta 837), reused 880 (delta 675), pack-reused 663 (from 2)
Receiving objects: 100% (1787/1787), 960.28 KiB | 8.14 MiB/s, done.
Resolving deltas: 100% (1138/1138), done.
PS C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\custom_nodes> |
```

Step 2: Go to your Documents\ComfyUI folder. In there, create a new text file using Notepad and when you save it, change the file extension from .txt to .yaml. This tells the system it's a configuration file rather than a regular text document. When Windows asks you to confirm the extension change, click yes. Optionally, you can create the text file through the command line.

```
PS C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\custom_nodes> cd C:\Users\LabUser\Documents\ComfyUI
PS C:\Users\LabUser\Documents\ComfyUI> notepad extra_model_paths.yaml
```



A prompt might pop up and signal you to confirm the creation of a file if you choose to create through CLI, select yes.

Once the file is created, open it and paste in the following:

Copy & Paste Text:

```
comfyui:
  base_path: C:\Users\LabUser\Documents\ComfyUI\
  checkpoints: models\checkpoints
  clip: models\clip
  clip_vision: models\clip_vision
  configs: models\configs
  controlnet: models\controlnet
  diffusers: models\diffusers
  diffusion_models: models\diffusion_models
  embeddings: models\embeddings
  gligen: models\gligen
  hypernetworks: models\hypernetworks
  loras: models\loras
  photomaker: models\photomaker
  style_models: models\style_models
  text_encoders: models\text_encoders
  unet: models\unet
  upscale_models: models\upscale_models
  vae: models\vae
  vae_approx: models\vae_approx

comfyui:
  base_path: C:\Users\LabUser\Documents\ComfyUI\
  checkpoints: models\checkpoints
  clip: models\clip
  clip_vision: models\clip_vision
  configs: models\configs
  controlnet: models\controlnet
  diffusers: models\diffusers
  diffusion_models: models\diffusion_models
  embeddings: models\embeddings
  gligen: models\gligen
  hypernetworks: models\hypernetworks
  loras: models\loras
  photomaker: models\photomaker
  style_models: models\style_models
  text_encoders: models\text_encoders
  unet: models\unet
  upscale_models: models\upscale_models
  vae: models\vae
  vae_approx: models\vae_approx
```

Make sure to swap out "LabUser" in the base path with your personal Windows username as it may differ. This file will act as a map that tells ComfyUI where all of your AI models are stored so it can find them when it needs them.

Step 3: Instead of manually typing the same launch commands every time, you should create two shortcut batch files that do it for you automatically.

In your Documents\ComfyUI folder, create a new text file and save it with a .bat extension. Again, confirm the extension change when Windows asks. This will be your **Master** startup script, which launches the primary GPU instance.

- Optionally, the file can be created through CLI with the following command.
 - o notepad C:\Users\LabUser\Documents\ComfyUI\start_master.bat
 - o Confirm when windows ask to create the file

```
PS C:\Users\LabUser\Documents\ComfyUI> notepad C:\Users\LabUser\Documents\ComfyUI\start_master.bat
PS C:\Users\LabUser\Documents\ComfyUI>
```

Open it and paste in:

```
@echo off
```

```
cd C:\Users\LabUser\Documents\ComfyUI
```

```
C:\Users\LabUser\Documents\ComfyUI\venv\Scripts\python.exe C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\main.py --listen --enable-cors-header --port 8188 --extra-model-paths-config C:\Users\LabUser\Documents\ComfyUI\extra_model_paths.yaml
```

```
Pause
```

Then create a second .bat file the same way — this one will be your **Worker** script for the second GPU.

Creation through CLI:

- notepad C:\Users\LabUser\Documents\ComfyUI\start_worker.bat
- Confirm when windows ask to create the file

```
PS C:\Users\LabUser\Documents\ComfyUI> notepad C:\Users\LabUser\Documents\ComfyUI\start_worker.bat
```

The key difference between the two is that the worker script includes set CUDA_VISIBLE_DEVICES=1, which tells the system to use your second GPU instead of the first one. You should double check which number corresponds to which GPU on your machine. To do this, open Task Manager and click on the Performance tab. That is where your GPUs will be listed.

Paste the following into the start_worker.bat file:

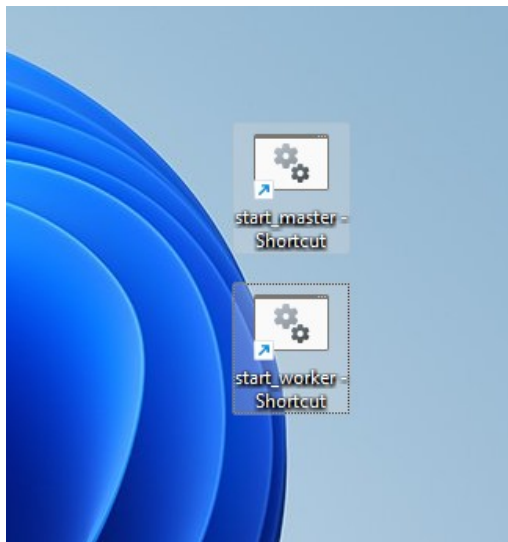
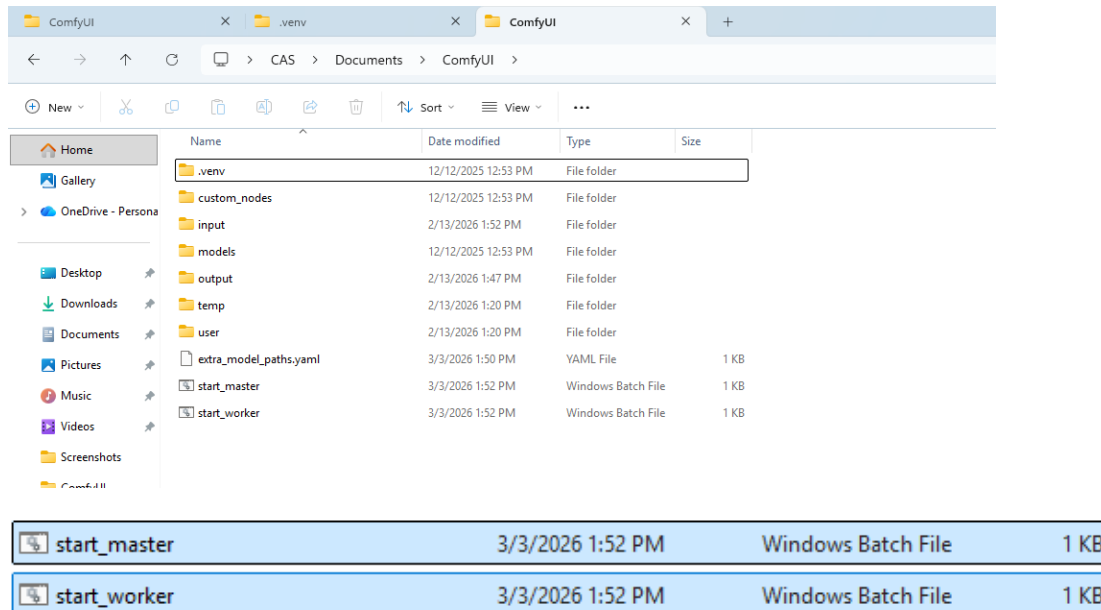
```
@echo off
```

```
cd C:\Users\LabUser\Documents\ComfyUI
```

```
set CUDA_VISIBLE_DEVICES=1
```

```
C:\Users\LabUser\Documents\ComfyUI\.venv\Scripts\python.exe C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\main.py --listen --enable-cors-header --port 8189 --extra-model-paths-config C:\Users\LabUser\Documents\ComfyUI\extra_model_paths.yaml
```

If you'd like easy access to these scripts, navigate to the location of your .bat files, then right-click each one and select "Create Shortcut," then drag those shortcuts to your desktop.



Step 4:

One important note is that you need to **launch the Master script first and wait for it to fully load before starting the Worker**. You'll know it's ready when the terminal shows a message along the lines of "To see the GUI go to

0.0.0.0:8188." Once you see that, go ahead and run the Worker script.
Launching both at the same time will cause them to crash.

```
start_worker - Shortcut
Device: cuda:0 NVIDIA RTX A4000 : cudaMallocAsync
Using async weight offloading with 2 streams
Enabled pinned memory 58738.0
working around nvidia conv3d memory bug.
Using pytorch attention
Python version: 3.12.11 (main, Aug 18 2025, 19:17:54) [MSC v.1944 64 bit (AMD64)]
ComfyUI version: 0.13.0
comfyui-frontend-package not found in requirements.txt
Failed to check frontend version: 'NoneType' object has no attribute 'split'
[Prompt Server] web root: C:\Users\LabUser\Documents\ComfyUI\.venv\Lib\site-packages\comfyui_frontend_package\static

Import times for custom nodes:
 0.0 seconds: C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\custom_nodes\websocket_image_save.py
 0.1 seconds: C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\custom_nodes\ComfyUI-Distributed

Context impl SQLiteImpl.
Will assume non-transactional DDL.
Assets scan(roots=['models']) completed in 0.056s (created=0, skipped_existing=22, orphans_pruned=0, total_seen=22)
Starting server

To see the GUI go to: http://0.0.0.0:8189
To see the GUI go to: http://[:]:8189

start_master - Shortcut
 0.1 seconds: C:\Users\LabUser\AppData\Local\Programs\ComfyUI\resources\ComfyUI\custom_nodes\ComfyUI-Distributed

Context impl SQLiteImpl.
Will assume non-transactional DDL.
Assets scan(roots=['models']) completed in 0.040s (created=0, skipped_existing=22, orphans_pruned=0, total_seen=22)
Starting server

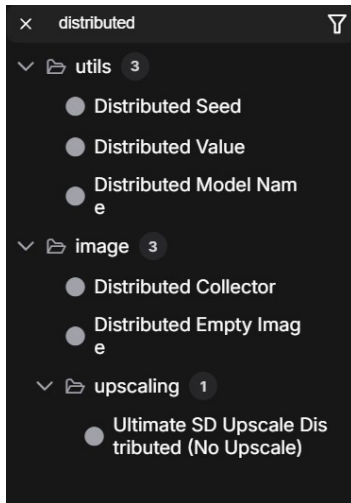
To see the GUI go to: http://0.0.0.0:8188
To see the GUI go to: http://[:]:8188
comfyui-frontend-package not found in requirements.txt
Exception in callback _ProactorBasePipeTransport._call_connection_lost(None)
handle: <Handle _ProactorBasePipeTransport._call_connection_lost(None)>
Traceback (most recent call last):
  File "C:\Users\LabUser\AppData\Roaming\uv\python\cpython-3.12.11-windows-x86_64-none\Lib\asyncio\events.py", line 88, in _run
    self._context.run(self._callback, *self._args)
  File "C:\Users\LabUser\AppData\Roaming\uv\python\cpython-3.12.11-windows-x86_64-none\Lib\asyncio\proactor_events.py", line 165, in _call_connection_lost
    self._sock.shutdown(socket.SHUT_RDWR)
ConnectionResetError: [WinError 10054] An existing connection was forcibly closed by the remote host
comfyui-frontend-package not found in requirements.txt
```

Once both are running, open the ComfyUI web interface in your browser. On the left side panel, you should now see a Distributed tab from the plugin you installed. Click "Add New Worker," name it "Worker 2," and set the worker type to **Local** rather than Remote. Selecting Local will reveal an extra field where you can enter the CUDA device number. Set this to 1 for your second GPU (or 2/3 if you wish to select a specific GPU to utilize).

The image displays two side-by-side screenshots of the 'Worker 2' configuration interface in ComfyUI. Both panels show the worker is 'Unconfigured remote worker • Port 8190' and has a 'View Log' button. The left panel shows the 'Remote' worker type selected, with fields for Name (Worker 2), Host (e.g., 192.168.1.100), and Port (8190). The right panel shows the 'Local' worker type selected, which reveals an additional 'CUDA Device' field set to 1. Both panels have 'Save', 'Cancel', and 'Delete' buttons at the bottom.

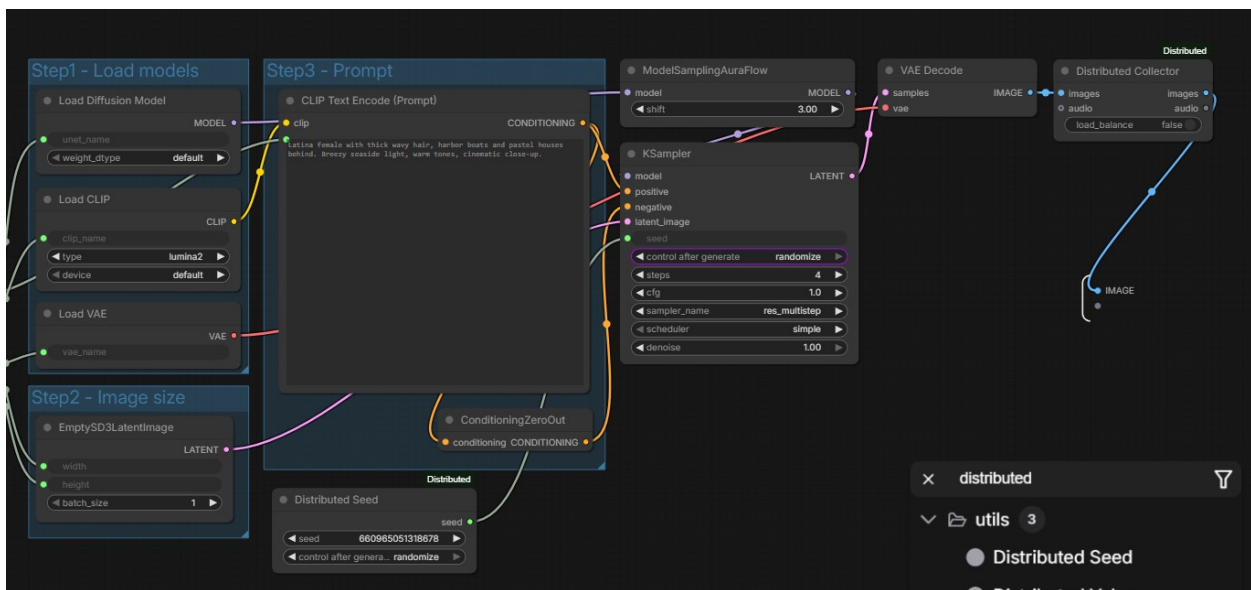
Now open the workflow you want to use for image generation. On the left panel, search for "Distributed" and you'll find two new nodes: the **Distributed Seed** and the **Distributed Collector**. Add both of them to your workflow canvas.

The Distributed Seed handles giving each GPU its own unique starting point for generation without this, both GPUs would just produce the same image. The Distributed Collector gathers the finished images from all GPUs and packages them together so you can see all of your outputs at once, making it easy to see the additionally created images.



With the distributed nodes installed, they can be put into any image generation workflow. It could be implemented into even the basic standard txt2img workflow template.

The following example was made following the z-turbo's Template:

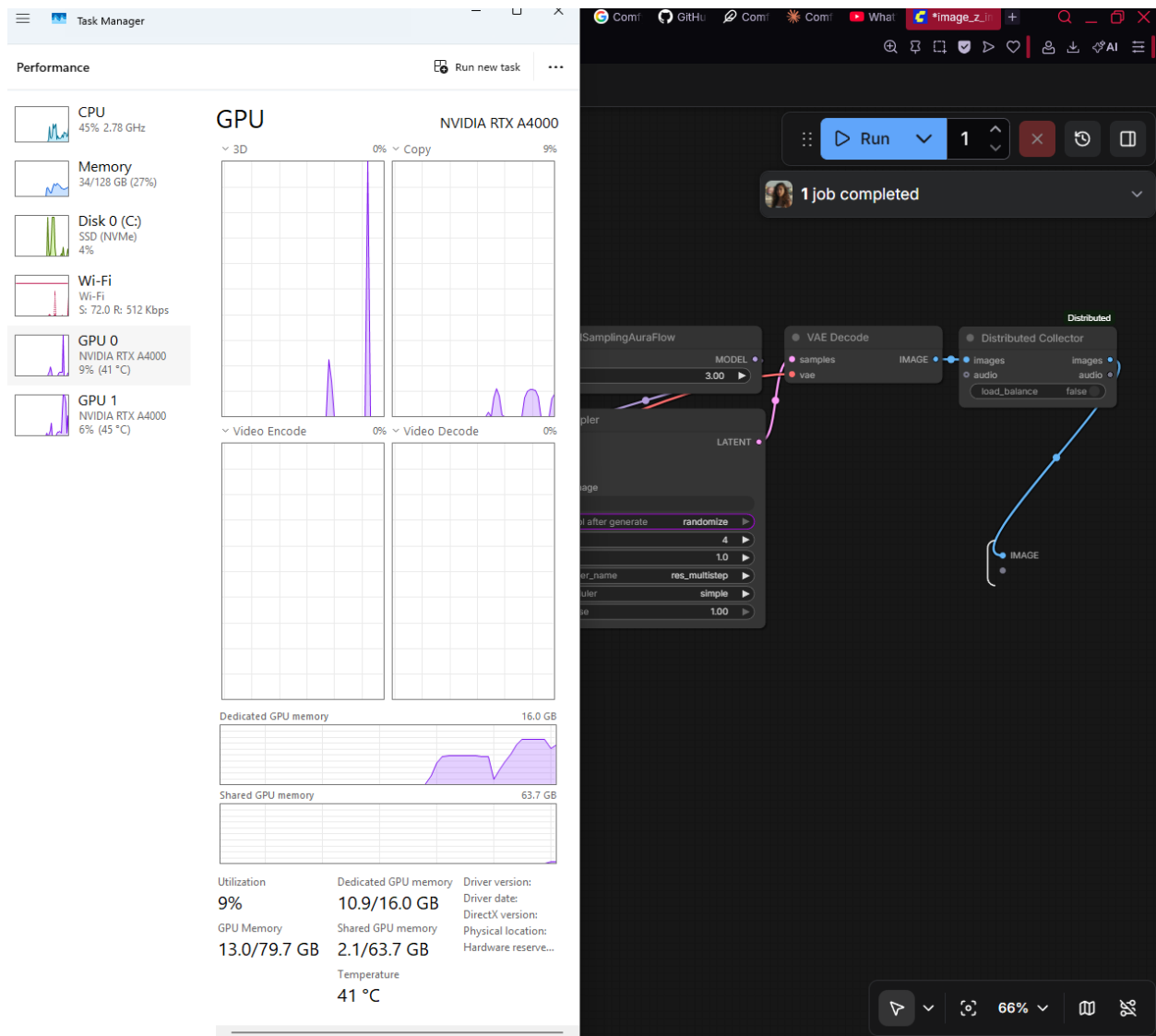


To complete the adaptation of multiGPU, connect the image output from your VAE Decode node into the Collector's image input, then connect the Collector's output to your final export node. Connect the Distributed Seed to the seed input on your K-Sampler node.

Now press the generate button and open task manager to see the RAM usage of both GPUs go to work. When the job finishes, you should receive a completion notification, and your output should show one image per GPU. If you're only getting one image while supposed to get multiple, go back to the

Distributed tab on the left and make sure both the master and worker are checked as active participants.

This is what it should look like when the PC is using both GPU's and what a completion of job looks like as a notification.



Keep watching the temperatures of various hardware while running jobs (namely the GPU), especially for tasks that require your system to run for an extended amount of time. Running multiple GPUs at full load generates a lot of heat, so make sure your system has enough cooling and that you're not overdrawing power and breaking your setup.

Conclusion:

Setting up a small server rack or desktop with a few high-end AI GPUs could fully suffice small group/company needs depending on the company size. Running it locally is completely feasible and reasonable even with limited investment and proves that you do not need to rely on large enterprise-level datacenters for AI in most cases.